ITI1120 LAB 2 : SOME EXERCISES

The following pages have a list of expressions. For each expression, first compute the expression in your head, without Python. Write down what you think the value is in the second column of the table. If you have no idea, write ?.

Next, use Python Interpreter to compute the same expression. Type each of these expressions one by one. You may find it easier to copy-and-paste them from this pdf file. Write down Python's result in the third column. You should always fill in the second and third column of a row before moving on to the next row. You want to learn from earlier examples before moving on to the next one.

If the two values are different, you should try to figure out why Python gave the answer that it did. Come up with a reasonable explanation and put it in the final column. You are not being graded on correctness in these labs so make your best guess at what is happening; your answer will help TA understand how to better aid you.

This lab is just supposed to be practice; do not waste too much time trying to figure things out yourself. If you do not understand something, ask TA immediately. It is very important that you understand how each expression is evaluated, so if an answer doesn't make sense, ask someone.

Do not forget to type import math before calling functions from math module

Expression Evaluation Table

Expression	Expected Value	Python Displayed Value	Reason for Displayed Value
2 * 3	6	6	Simple multiplication of two integers. Human expectation matches the displayed result.
2 * 3.0	6	6.0	Humans might expect 6, but since one operand is a float, Python returns a float result.
2 ** 3	8	8	Exponentiation of 2 raised to the power of 3.
2 * 3 - 2/2	5	5.0	Humans would expect integer results for simple arithmetic like 6 - 1 = 5, but Python returns a float due to the division (2/2 = 1.0).
16/(4/(1/2))	2	2.0	Humans expect integer division as 16 ÷ (4 ÷ 0.5) = 2, but Python's division returns a float.
16/4/(1/2)	2	2.0	Human expectation is that dividing 16 by 4 gives 4, then dividing by 0.5 gives 2, but Python returns a float result.
16/(1+2)+2)	Error	Syntax Error	Mismatched parenthesis, resulting in a syntax error in Python.
2 * 3**2	18	18	Human expectation and Python match: 2 * 9 = 18.
(2 * 3)**2	36	36	Human expectation matches Python: 6 squared is 36.
2 ** 2 ** 0	2	2	Python's exponentiation is evaluated right to left, resulting in 2. Human expectation matches the result.
(2 ** 2) ** 0	1	1	Human expectation matches Python: any number raised to the power of 0 is 1.
-4 - -4 - -4	4	4	Human expectation and Python match: two negatives in subtraction cancel each other out.
6/2	3	3.0	Humans would expect integer division, but Python returns a float for division.
6/4	1	1.5	Humans might expect floor division (resulting in 1), but Python returns a floating-point result of 1.5.
6//4	1	1	Human expectation and Python match, as // is integer (floor) division.
6 % 3	0	0	Human expectation and Python match: remainder of 6 divided by 3 is 0.
7 % 2	1	1	Human expectation and Python match: remainder of 7 divided by 2 is 1.
6.2 % 4	2.2	2.2	Human expectation and Python match: remainder of 6.2 divided by 4 is 2.2.
5 < 3	False	False	Human expectation and Python match: 5 is not less than 3.

Table 2: Function Calls with Human Expectations

Expression	Expected Value	Python Displayed Value	Reason for Displayed Value
min(25, 4)	4	4	The min() function returns the smallest of the two values, which is 4.
max(25, 4)	25	25	The max() function returns the largest of the two values, which is 25.
min(25, max(27, 4))	25	25	First, max(27, 4) evaluates to 27, and then min(25, 27) evaluates to 25.
abs(25)	25	25	The absolute value of 25 is 25.
abs(-25)	25	25	The absolute value of -25 is 25.
round(25.6)	26	26	Humans would expect rounding 25.6 to the nearest integer, which is 26.
round(-25.6)	-26	-26	The round() function rounds -25.6 to the nearest integer, which is -26.
round(25.64, 0)	26	26	Rounding 25.64 to 0 decimal places results in 26.
round(25.64, 1)	25.6	25.6	Rounding to 1 decimal place gives 25.6.
round(25.64, 2)	25.64	25.64	No change, as it already has 2 decimal places.
math.ceil(3.7)	4	4	Human expectation and Python match: rounding up 3.7 results in 4.
math.floor(3.7)	3	3	Human expectation and Python match: rounding down 3.7 results in 3.
type(4)	int	int	Humans expect the type of 4 to be an integer.
type(7//4)	int	int	Human expectation and Python match: integer division (//) returns an integer.
type(7/4)	float	float	Humans would expect normal division (/) to return a float, as the result is not a whole number.

Variables and Assignment Statements

The part involves assignment statements. You need to know the difference between expressions, which you've been working with so far, and assignment statements. Here is an example of an assignment statement:

b = 3 < 5

In particular, the above assignment statement is executed by Python as follows:

1. Python evaluates the expression on the right-hand side of the = (in this case, 3 < 5), and
2. then it makes the variable on left-hand side of the =, in this case, b, refer to this value.

Because it is not an expression but rather an assignment statement, Python Interpreter will not actually output a result when you type it in; it will just perform the command silently. In the table below, the first column contains either an expression or an assignment statement. If it is an expression, write the value. If it is an assignment statement, you should just write Nothing (we have done the first one for you). Because some of the entries are assignment statements, it is important that you enter the expressions or assignment statement, in exactly the order they are given.

Table 3: Variables and Assignment Statements

Statement or Expression	Expected Value	Python Displayed Value	Reason for Displayed Value
i = 2	Nothing	Nothing	Assignment statements do not produce output.
i	2	2	The value of `i` is 2 after the assignment.
j	Error	Error/Not Defined	`j` has not been assigned a value yet.
j = 1	Nothing	Nothing	Assignment of 1 to `j`, no output expected.
j	1	1	The value of `j` is now 1.
j = j + i	3	3	`j` is updated to `j + i`, which is 1 + 2 = 3.
j	3	3	The value of `j` is 3 after the update.
i	2	2	The value of `i` remains 2.
w = 'Hello'	Nothing	Nothing	Assignment of the string 'Hello' to variable `w`.
i + w	Error	TypeError	Adding an integer (`i`) to a string (`w`) is not allowed, resulting in a TypeError.

a = 1 == 2	False	False	`1 == 2` evaluates to `False`, which is then assigned to `a`.
a	False	False	The value of `a` is `False`.
a = min(pow(2,3,2), 2)	0	0	`pow(2, 3, 2)` returns the remainder of `2^3` divided by 2, which is `0`. `min(0, 2)` results in 0.
a	0	0	The value of `a` is now 0.